64. iAP2

iAP2 is a complete replacement for iAP1 and is not backward compatible.

64.1 iAP2 Connection

An iAP2 *connection* is composed of an iAP2 *link* over an iAP2 *transport* and one or more iAP2 *sessions*.

64.2 iAP2 Link

Every iAP2 connection starts with the establishment of a link between an accessory and a device over a supported transport. The link protocol provides a transport-agnostic mechanism for reliable and ordered delivery of packetized data belonging to one or more iAP2 sessions. The protocol is also configurable on a per-connection basis and can be optimized for best performance on any particular transport and accessory usage profile. Certain protocol features contribute towards these goals:

- Positive acknowledgement of received packets.
- Retransmissions only require the unacknowledged packets in a sequence to be resent.
- Explicit and efficient support for iAP2 sessions.

iAP2 is supported over the following transports:

- Bluetooth (page 525).
- UART (page 505).
- USB Device Mode (page 513).
- USB Host Mode (page 510).
- Apple Lightning Audio (page 504).

All devices supporting one or more transports can be used to establish an iAP2 connection with an accessory. Some transports are better suited for various features than others, so transport selection should be a high priority early in the accessory design process.

64.2.1 Packet Structure

Every link packet shall start with a fixed-size 9-byte header, including checksum, followed by an optional variable-length data payload. Both the header and payload have their own checksums. If there is no payload data, there is no payload checksum. An example can be found in iAP2 Link Packet Structure Example (page 474).

Table 64-1 iAP2 Link Packet Structure

Start of Packet MSB (`0xFF`)
Start of Packet LSB (`0x5A`)
Packet Length MSB
Packet Length LSB
Control Byte
Packet Sequence Number
Packet Acknowledgement Number
Session Identifier
Header Checksum
...
Payload Data
...
Payload Checksum

64.2.1.1 Start of Packet

The first two bytes of every iAP2 link packet are always `FF 5A`. If these bytes are detected in the transport stream then both accessory and device shall attempt to parse the following bytes as a valid packet.

64.2.1.2 Packet Length

The next two bytes denote the Packet Length in bytes and are always expressed as an unsigned 16-bit big-endian integer. A link packet with no Payload Data always has a Packet Length of 9 bytes (from Start of Packet to Header Checksum). Otherwise, the Packet Length is measured from the Start of Packet to the last byte of Payload Data including the Payload Checksum. For example, a link packet with 1 byte of Payload Data has a Packet Length of 11 bytes. Packets have a maximum Payload Data size of 65525 bytes; a packet with such a payload has a Packet Length of 65535 bytes.

64.2.1.3 Control Byte

The bits in the Control Byte indicate what is present in the packet.

- The SYN, EAK, and RST bits are mutually exclusive.
- The ACK bit may be combined with the SYN bit.
- The EAK bit is always set along with the ACK bit.
- The RST bit is never set in conjunction with any other bit.
- The SLP bit is never set in conjunction with any other bit.
- All unassigned bits shall be set to 0.

iAP2 Session Payloads cannot be present when any of the SYN, EAK, or RST bits are set. Conversely, link packets with iAP2 session payloads always have the ACK bit set.

Table 64-2 iAP2 Link Control Byte Bits

Bit	Name	Meaning
7	SYN	Link Synchronization Payload is present
6	ACK	Packet Acknowledgement Number is valid, and iAP2 Session Payload may be present
5	EAK	Extended Acknowledgement Payload is present
4	RST	Link Reset
3	SLP	Device Sleep

In the rest of this specification, a link packet can be described in terms of its Control Byte bits and payload. For example, a SYN packet refers to a link packet with the SYN bit set in the Control Byte, and a SYN+ACK packet refers to a link packet with both the SYN and ACK bits set in the Control Byte.

64.2.1.4 Packet Sequence Number

Every link packet contains a Packet Sequence Number uniquely identifying the packet among all other packets in transit. When a link is first created, both the device and accessory shall randomly pick an initial sequence number before sending their first SYN/SYN+ACK packet. Each time a packet with iAP2 Session or Link Synchronization Payload Data is sent, the sequence number is incremented by 1. Otherwise, the sequence number does not increment when a packet is sent. Retransmissions of a previously sent packet shall retain the same Packet Sequence Number. The sequence number wraps back to `0` upon reaching 255.

64.2.1.5 Packet Acknowledgement Number

The Packet Acknowledgment Number only has meaning if the ACK bit in the Control Byte is set. If ACK is not set, the Packet Acknowledgement Number shall be set to `0` by the sender, and it shall be ignored by the receiver.

If ACK is set, the Packet Acknowledgment Number indicates to a sender the Packet Sequence Number of the last in-sequence packet received. For example, if the Packet Sequence Numbers 1, 2, 3, and 5 were received by the accessory, the accessory's next outgoing packet's Packet Acknowledgement Number would be 3, because 5 was received out of sequence.

64.2.1.6 Session Identifier

The Session Identifier only has meaning if the ACK bit in the Control Byte is set and there is an iAP2 session payload present. If those two conditions are met, the Session Identifier will be a nonzero number specifying a particular session in an iAP2 connection. Otherwise, the Session Identifier shall be set to `0`.

64.2.1.7 Header Checksum

The Header Checksum is calculated by adding together all of the following packet bytes. If the value in the Header Checksum does not match the value calculated according to Checksum Calculation (page 457), the receiver shall restart packet parsing from the next detected Start of Packet sequence.

- Start of Packet MSB
- Start of Packet LSB
- Packet Length MSB
- Packet Length LSB
- Control Byte
- Packet Sequence Number
- Packet Acknowledgement Number
- Session Identifier

64.2.1.8 Payload Data

This section is optional, and its presence shall match the state of the bits in the Control Byte. The maximum possible payload size is 65,525 bytes.

64.2.1.9 Payload Checksum

The Payload Checksum byte is present if and only if Payload Data is present. If Payload Data is present, all of the bytes of the Payload Data are checksummed. If the value in the Payload Checksum does not match the value calculated according to Checksum Calculation (page 457), the receiver shall restart packet parsing from the next detected Start of Packet sequence.

64.2.1.10 Checksum Calculation

A checksum byte, as used by iAP2, is calculated for each packet sent. The intent is for the sum of all (unsigned 8-bit) bytes being checksummed and the checksum (unsigned 8-bit) byte, ignoring any unsigned 8-bit overflow, to be equal to `0x00`. This allows a fast way to verify the packet was correctly transmitted. The checksum byte is calculated by taking the least significant byte of the sum of the (unsigned 8-bit) bytes being checksummed, and taking the two's complement of the 8-bit value. Example code is included below:

```
uint8_t
checksum_calculation(uint8_t *buffer, uint16_t start, uint16_t length)
{
    uint16_t i;
    uint8_t  sum = 0;
    for (i = start; i < (start + length); i++) {
        sum += buffer[i];
    }
    return (uint8_t)(0x100 - sum); /* 2's complement */
}
```

64.2.2 Link Synchronization Payload

The Link Synchronization Payload (LSP) is used to establish a link and synchronize Packet Sequence Numbers between the device and accessory. It also contains the negotiable link parameters. An example can be found in iAP2 Link Synchronization Payload Example (page 474).

Table 64-3 Link Synchronization Payload (Version 1)

Link Version (0x01)
Maximum Number of Outstanding Packets
Maximum Received Packet Length MSB
Maximum Received Packet Length LSB
Retransmission Timeout MSB
Retransmission Timeout LSB
Cumulative Acknowledgement Timeout MSB
Cumulative Acknowledgement Timeout LSB
Maximum Number of Retransmissions
Maximum Cumulative Acknowledgements
iAP2 Session 1: Session Identifier
iAP2 Session 1: Session Type
iAP2 Session 1: Session Version
...
iAP2 Session N: Session Identifier
iAP2 Session N: Session Type
iAP2 Session N: Session Version

64.2.2.1 Link Version

- The version of the link being established. All packet payloads may vary depending on the Link Version.
- The only valid value of Link Version at this time is 1.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.2.2 Maximum Number of Outstanding Packets

- The maximum number of packets sent without receiving an acknowledgement.
- Valid values are 1 to 127.
- This is not a negotiable parameter.
- The accessory and device may propose and use different values.
- The accessory shall not send more than the device's proposed Maximum Number of Outstanding Packets without waiting for an acknowledgement from the device, and vice versa.

64.2.2.3 Maximum Received Packet Length

- The largest possible Packet Length in bytes.

- Valid values are 24 to 65535.
- This is not a negotiable parameter.
- The accessory and device may propose and use different values.

64.2.2.4 Retransmission Timeout

- The timeout value in milliseconds for retransmission of unacknowledged packets. This should be set to a value approximating the transmission time for a packet over the link transport.
- Valid values are 20 ms to 65535 ms.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.2.5 Cumulative Acknowledgement Timeout

- The timeout value in milliseconds after which an acknowledgment packet shall be immediately sent if another packet is not sent.
- Valid values are 10 ms to half of the Retransmission Timeout.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.2.6 Maximum Number of Retransmissions

- The maximum number of packet retransmissions attempted before the link is considered to be broken.
- Valid values are 1 to 30.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.2.7 Maximum Cumulative Acknowledgements

- The maximum number of received sequence numbers accumulated before acknowledgement shall occur. See Acknowledgements (page 465) and Flow Control (page 466).
- Valid values are 0 to 127 or the Maximum Number of Outstanding Packets, whichever is smaller.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.2.8 ZeroACK/ZeroRetransmit Link Configurations

Starting in iOS 8.3, accessories may choose to try and negotiate a ZeroACK/ZeroRetransmit link configuration without expecting retransmitted packets or packet acknowledgements. This is only recommended if the underlying transport has its own mechanisms for reliable delivery and the accessory fully utilizes those mechanisms. iAP2 connections over USB Host Mode (page 510), USB Device Mode (page 513), or Bluetooth RFCOMM transports are possible candidates.

Devices will still detect dropped link packets using packet sequence numbers and reset the iAP2 link. Accessories shall not use this link configuration if dropped packets regularly occur during typical usage, and shall be prepared to re-establish the link when a reset occurs without losing state or compromising user functionality.

- When the device detects a dropped packet, a RST will be sent to the accessory and link layer negotiation will restart with a SYN packet from the accessory.
- When the accessory detects a dropped packet using packet sequence numbers, the accessory shall send a SYN packet to restart link negotiation

To negotiate a zero-acknowledgement/zero-retransmit link configuration, the following negotiable link parameters shall be set to 0:

- Retransmission Timeout
- Cumulative Acknowledgement Timeout
- Maximum Number of Retransmission
- Maximum Cumulative Acknowledgements

Some devices and/or iOS releases may reject attempts to negotiate a ZeroACK/ZeroRetransmit link configuration. Accessories shall retain the capability to negotiate a link with acknowledgements and retransmissions if they claim compatibility with those devices/iOS releases. The accessory shall send a SYN packet, not SYN+ACK, with updated parameters to continue negotiation.

Accessories shall account for the lack of flow control at the link layer by managing their iAP2 session traffic. For example, MediaLibraryUpdate messages should be paused by sending a StopMediaLibraryUpdate before starting a file transfer or vice versa, and resume the other activity when appropriate. The same caution applies to External Accessory Protocol messages.

64.2.2.9 iAP2 Sessions

- The iAP2 sessions the accessory will use to communicate with the device.
- Session identifiers shall be unique to each defined session. 0 is not a valid session identifier.
- Session Type (page 476) and Version (page 476) shall be valid.
- This is a negotiable parameter.
- Both the accessory and device shall agree on the same value.

64.2.3 iAP2 Session Payload

The iAP2 Session Payload is specified in iAP2 Sessions (page 475). The ACK bit in the Control Byte shall be set whenever an iAP2 Session Payload is present.

64.2.4 Extended Acknowledgement Payload

The Extended Acknowledgement Payload is used to acknowledge packets received out of sequence. This payload has the following attributes:

- Both the EAK and ACK bits in the Control Byte shall be set.
- The Packet Acknowledge Number contains the sequence number of the last packet received in sequence.
- The Payload Data section contains the sequence numbers of one or more packets received out of sequence. They are not acknowledgements for those packets. Acknowledgements will be sent separately in a later packet with the appropriate number in the ACK field.

Table 64-4 EAK Packet Payload (Link v1)

1st Out of Sequence Acknowledgement Number
...
Nth Out of Sequence Acknowledgement Number

64.2.5 Reset

The RST bit in the Control Byte is used by the device to reset a connection. Such a link packet does not have a payload and may only be sent by the device.

64.2.6 Sleep

This section only applies to accessories integrating the following Lightning connectors:

- Lightning (C78-USBH)
- Lightning (C79-USBH)
- Lightning (C79-UART)
- Lightning (C79-LA)
- Lightning (C78-STROBE-USBH)
- Lightning (C78-STROBE-UART)
- Lightning (C78-STROBE-LA)

The SLP bit in the Control Byte is used by the device to signal when it is about to enter a sleep state. Such a link packet does not have a payload and may only be sent by the device.

The device will stop supplying Accessory Power once it enters the sleep state. If the device exits the sleep state, it will start supplying Accessory Power again.

If the accessory is capable of suspend and resume of the link layer, the accessory shall wait 500 ms after Accessory Power returns before re-initializing the link. During this time, the accessory may receive an ACK from the device which signals a link layer resume and serves to synchronize the sequence number (last number sent from the device) and ACK number (the last number received by the device).

If the accessory is not capable of suspend and resume of the link layer, the accessory shall wait 80 ms after Accessory Power returns before re-initializing the link.

64.2.7 Operation

64.2.7.1 Record

All link implementations shall store the following variables in a link-specific record. The variables will be referred to in subsequent sections describing iAP2 Link operation.

Table 64-5 iAP2 Link Operation Record Variables

Variable	Description
SentACKTimer	A timer keeping track of the elapsed time (ms) since the last ACK packet was sent
NextSentPSN	The Packet Sequence Number of the next packet to be sent
OldestSentUnacknowledgedPSN	The Packet Sequence Number of the oldest unacknowledged packet
InitialSentPSN	The Packet Sequence Number used for the very first packet sent
LastReceivedInSequencePSN	The Packet Sequence Number of the last packet received correctly and in sequence
InitialReceivedPSN	The Packet Sequence Number of the very first packet received
ReceivedOutOfSequencePSNs[n]	An array of Packet Sequence Numbers received and acknowledged out of sequence

64.2.7.2 Initialization

Once the transport connection is established, the accessory shall confirm the presence of a device supporting iAP2 by sending the following byte sequence at 1 Hz (once a second) until a response is received from the device: `FF 55 02 00 EE 10`.

If the device supports iAP2, the accessory will receive the same byte sequence in return.

If the device only supports iAP1, the accessory will receive one of the following:

- `55 04 00 02 04 EE 08`
- `55 02 00 00 FE`
- `FF 55 04 00 02 04 EE 08`
- `FF 55 02 00 00 FE`

If the accessory receives one of those responses, it shall send the following byte sequence to the device to indicate lack of compatibility: `FF 55 0E 00 13 FF FF FF FF FF FF FF FF FF FF FF FF EB`.

The accessory shall treat any other response from the device as final; no retries or retransmissions are allowed.

64.2.7.3 Synchronization

One notable feature of the link is support for automatic negotiation of transmission parameters over any type of transport. This permits the link to scale according to the capabilities of both device and accessory. Link initialization has 3 main goals:

- Determines mutually acceptable link configuration parameters for both device and accessory.
- Searches for errors in configuration parameters.
- Terminates the link if no agreement can be reached.

A link is initiated when an underlying transport connection and device compatibility have been established. When this occurs, the accessory first sends a SYN packet containing desired connection parameters in the Link Synchronization Payload, along with a randomly generated Packet Sequence Number. The device will respond with an SYN+ACK packet acknowledging receipt of the accessory's SYN packet, its own desired connection parameters, and its randomly generated Packet Sequence Number.

If the device and accessory have proposed identical values for all negotiable connection parameters, the accessory shall send a final ACK packet of the most recent SYN+ACK from the device, and the connection is considered to be established. Otherwise, SYN+ACK packets will continue to be exchanged up to `10` times until all negotiable connection parameters are agreed upon. If `10` exchanges occur without agreement on negotiable connection parameters the device will stop responding to any further packets sent by the accessory.

Sometimes, a device will be unable to respond immediately to the receipt of a SYN packet from an accessory. The accessory may resend the SYN packet with the same Payload Data and the same Packet Sequence Number. The accessory may continue doing so every second until the device sends a SYN+ACK response acknowledging the previously sent Packet Sequence Number. Once the device response has been received, the accessory shall ignore any subsequent incoming SYN+ACK packets with the same Packet Sequence Number.

The device will send a RST packet to reset the link if the accessory's final Link Synchronization Payload contains an invalid non-negotiable parameter.

During synchronization, the following default link configuration parameters are assumed until synchronization is complete:

Table 64-6 Default link parameters during synchronization

Parameter	Default Value
Maximum Number of Outstanding Packets	1
Maximum Received Packet Length	128 bytes
Retransmission Timeout	1000 ms
Cumulative Ack Timeout	10 ms
Maximum Number of Retransmissions	30
Maximum Cumulative Acknowledgements	0

The following tables contain recommended link configuration parameters for some iAP2 transports. These values are just starting points for development and shall not be used without verifying suitability for a particular accessory.

Table 64-7 Recommended link parameters for USB Host Mode transport (Full Speed)

Parameter	Recommended Value
Maximum Number of Outstanding Packets	5
Maximum Received Packet Length	4096 bytes
Retransmission Timeout	2000 ms
Cumulative Ack Timeout	22 ms
Maximum Number of Retransmissions	30
Maximum Cumulative Acknowledgements	3

Table 64-8 Recommended link parameters for USB Device Mode transport (Full Speed)

Parameter	Recommended Value
Maximum Number of Outstanding Packets	5
Maximum Received Packet Length	4096 bytes
Retransmission Timeout	2000 ms
Cumulative Ack Timeout	22 ms
Maximum Number of Retransmissions	30

Parameter	Recommended Value
Maximum Cumulative Acknowledgements	3

Table 64-9 Recommended link parameters for Bluetooth transport

Parameter	Recommended Value
Maximum Number of Outstanding Packets	5
Maximum Received Packet Length	2048 bytes
Retransmission Timeout	1500 ms
Cumulative Ack Timeout	73 ms
Maximum Number of Retransmissions	30
Maximum Cumulative Acknowledgements	3

Table 64-10 Recommended link parameters for 115.2 kbps UART transport

Parameter	Recommended Value
Maximum Number of Outstanding Packets	4
Maximum Received Packet Length	256 bytes
Retransmission Timeout	1000 ms
Cumulative Ack Timeout	133 ms
Maximum Number of Retransmissions	30
Maximum Cumulative Acknowledgements	2

64.2.7.4 Acknowledgements

To guarantee delivery of packets, the link protocol uses both positive acknowledgement and retransmission of packets. Each packet with Payload Data and SYN packets are acknowledged when they are correctly received and accepted by the receiver. ACK only packets are not acknowledged. Damaged packets are discarded and not acknowledged. The sender shall retransmit any packets that are not acknowledged by the receiver within the time specified by the Retransmission Timeout.

There are two types of packet acknowledgements. A cumulative acknowledgement is used to acknowledge receipt of all in-sequence packets up to a specified Packet Sequence Number. The extended acknowledgement allows the receiver to acknowledge packets out of sequence. The type of acknowledgement used is simply a function of the order in which packets arrive. Whenever possible, link implementations shall use the cumulative acknowledgement and use the extended acknowledgement only when packets are received out of sequence.

Receivers shall generate and send an extended acknowledgement as soon as one or more out-of-sequence packets are received.

The same Payload Data may be received multiple times if the corresponding ACK is lost in transmission. Receivers shall resend the ACK for the Payload Data when this occurs.

64.2.7.5 Retransmissions

Packets can get lost in transmission because of loss or damage in the underlying transport. They can also be discarded by the receiver. The positive acknowledgement of packets requires the receiver to acknowledge a packet only when the packet has been correctly received and accepted.

To detect missing packets, the sender shall use a retransmission timer for each outgoing packet. This timer is set to the negotiated Retransmission Timeout value. When an acknowledgement is received for a packet, the timer is cancelled for the packet. If the timer expires before an acknowledgement is received, the packet is retransmitted and the timer is restarted, up to the Maximum Number of Retransmissions negotiated during link synchronization.

64.2.7.6 Flow Control

The iAP2 link employs a flow control mechanism based on the number of unacknowledged packets sent and the Maximum Number of Outstanding Packets link configuration parameter. This parameter is specified by each side when the link is created, and should be set based on the number and size of buffers each side is willing to allocate to the link. Once set the parameter remains unchanged while connected.

The link employs the concept of a sequence number window for acceptable Packet Sequence Numbers. The left edge of the window is the last in-sequence acknowledged Packet Sequence Number plus one. The right edge of the window is equal to the last in-sequence acknowledged Packet Sequence Number plus the Maximum Number of Outstanding Packets. The link sender sends packets until the receiver's Maximum Number of Outstanding Packets is reached. Once the limit is reached the sender may only send a new packet for each acknowledged packet.

When a received packet has a Packet Sequence Number falling within the window, it is acknowledged. If the Packet Sequence Number is equal to the left edge (that is, it is the next expected Packet Sequence Number), the packet is acknowledged with a cumulative acknowledgement (ACK), and the acceptance window's left and right edges are incremented by one. If the received packet's Packet Sequence Number is within the window but out of sequence, it is acknowledged with an Extended Acknowledgement (EAK). The window is not adjusted, and the receipt of the out-of-sequence packet is recorded. Received link packets with Packet Sequence Numbers outside the window shall be discarded; and are an indication the link is unstable and may need to be reset.

When a sender receives Extended Acknowledgements (EAK), the sender shall not transmit beyond the acceptance window. This could occur if one packet is not acknowledged but all subsequent packets are received and acknowledged. This requirement will fix the left edge of the window at the sequence

number of the unacknowledged packet. As additional packets are sent, the next Packet Sequence Number will approach and eventually overtake the right edge. At this point, no more packets may be sent until the unacknowledged packet is acknowledged.

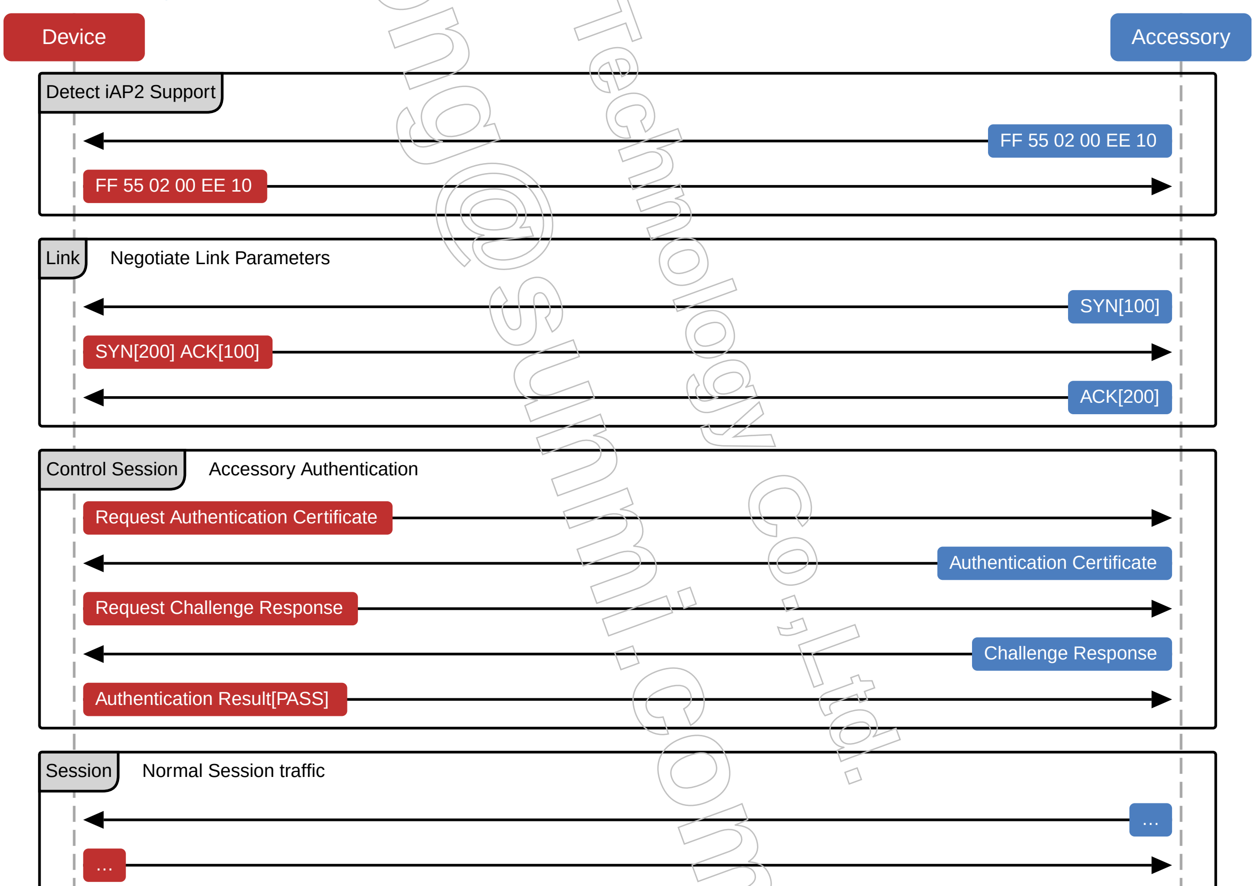
64.2.7.7 Reset

The device may send a RST packet to the accessory at any time. Upon receipt of an RST packet, the accessory shall send a SYN packet to the device within 1 second of receiving the RST packet and then proceed with Synchronization (page 463).

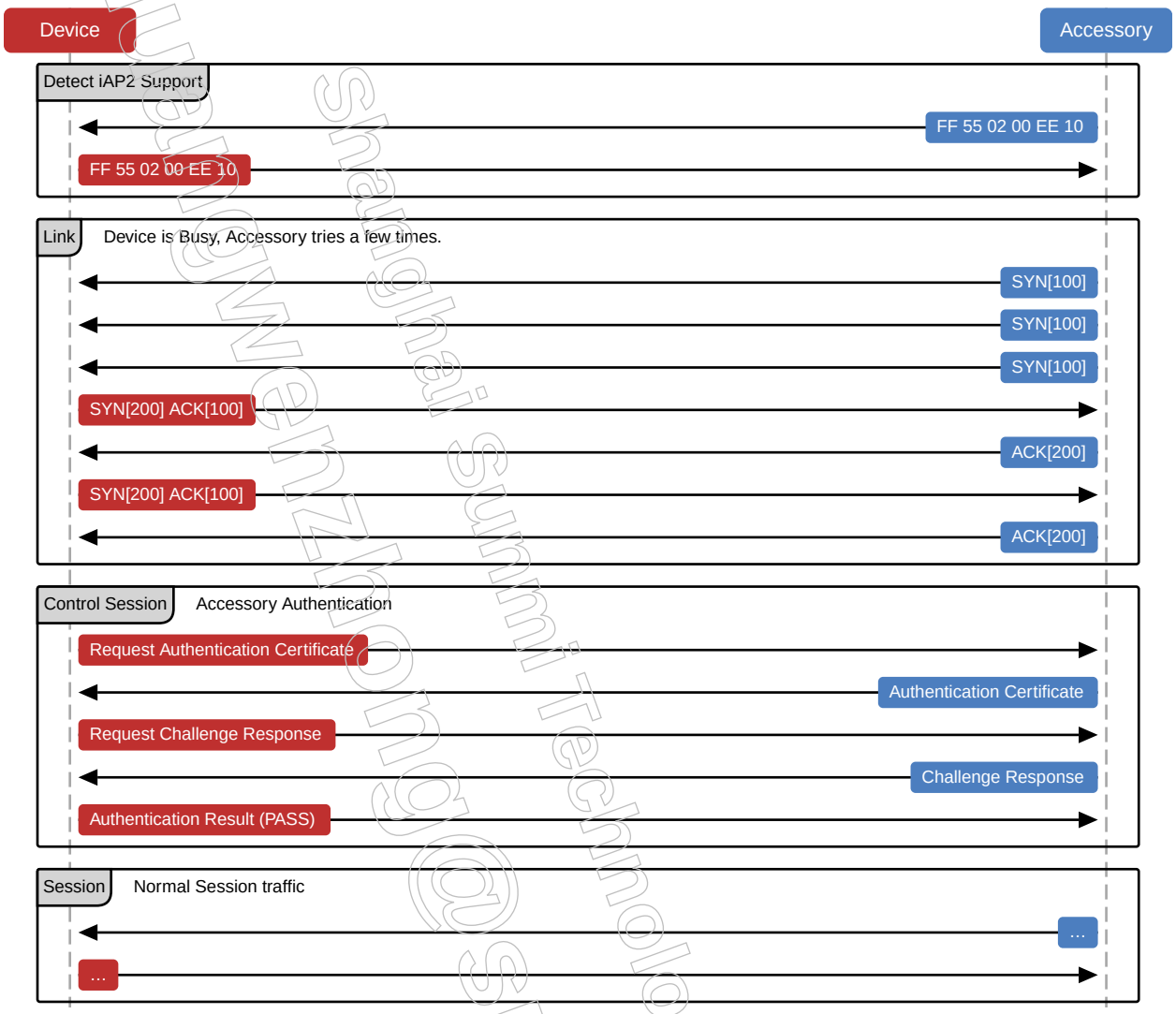
Accessories shall terminate the underlying transport connection and reconnect if they wish to reset the link. This is not possible when the UART transport is active without physically disconnecting and reconnecting the accessory with the device's Lightning connector. Having to restart the link is a sign of poorly negotiated link parameters and should not occur during typical operation.

64.2.8 Examples

64.2.8.1 Typical Link Initialization

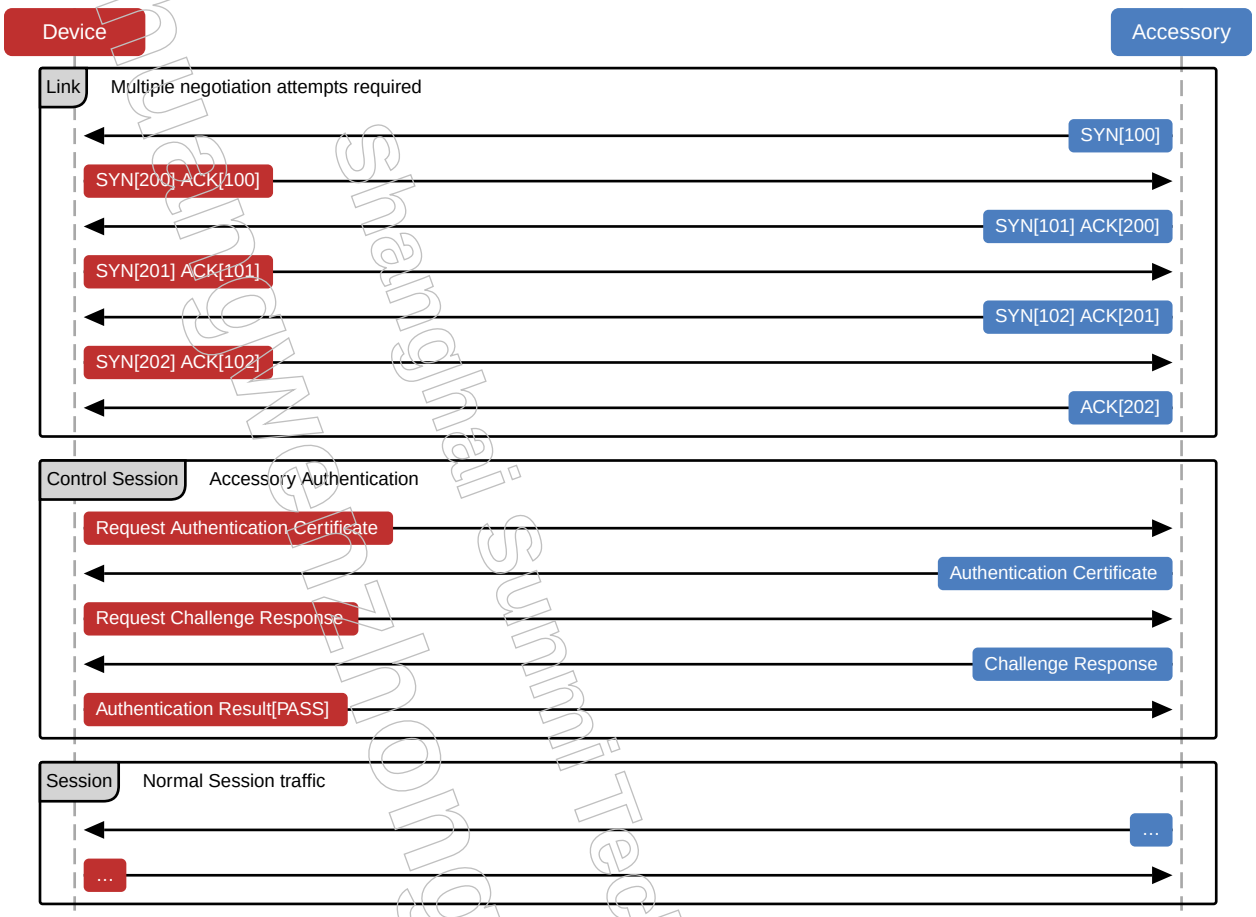


64.2.8.2 Connection Initialization When Device is Busy

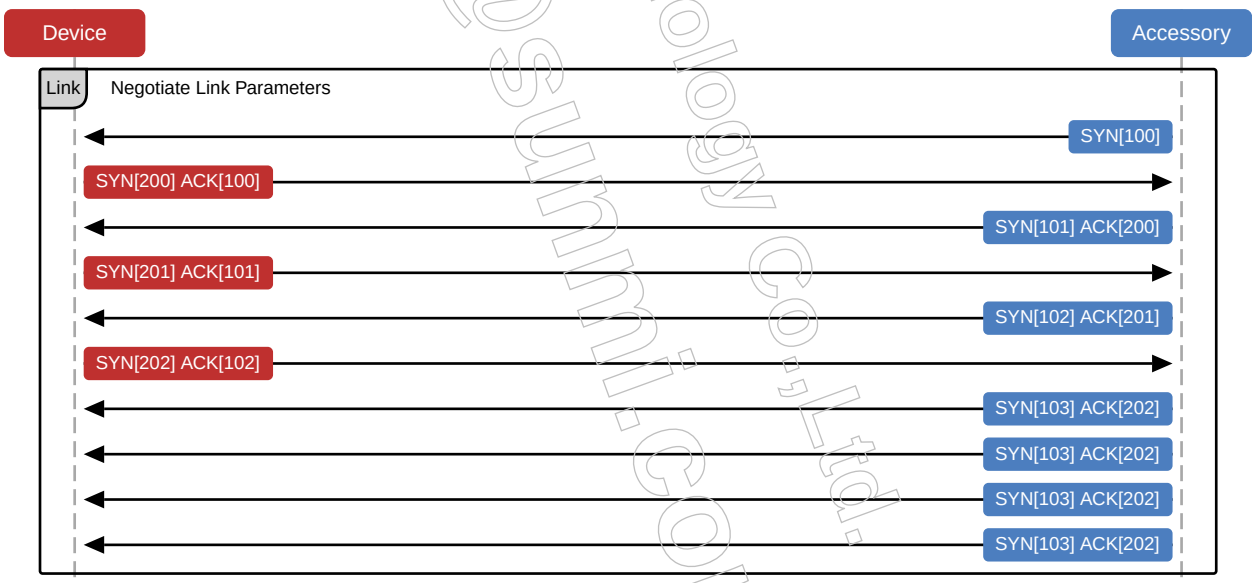


64.2.8.3 Connection Requiring Multiple Negotiation Attempts

Multiple negotiation attempts may occur when an accessory and a device do not mutually agree on iAP2 Link parameters.



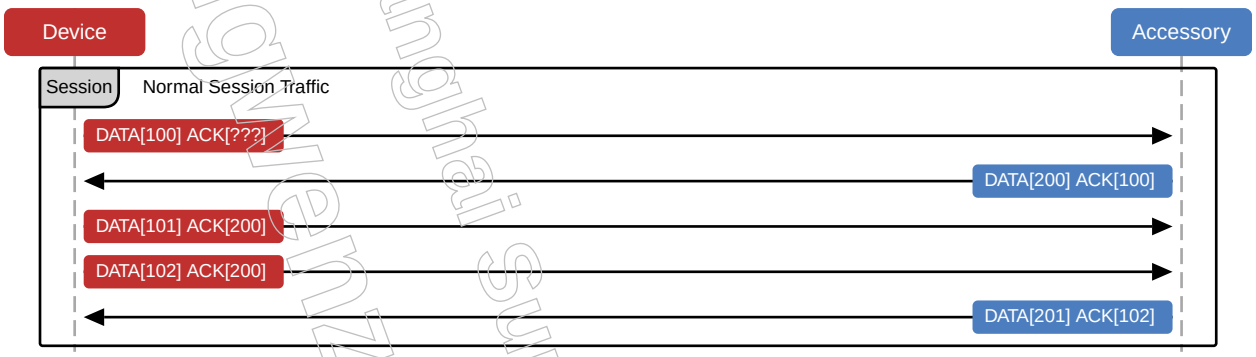
64.2.8.4 Connection With Failed Negotiation



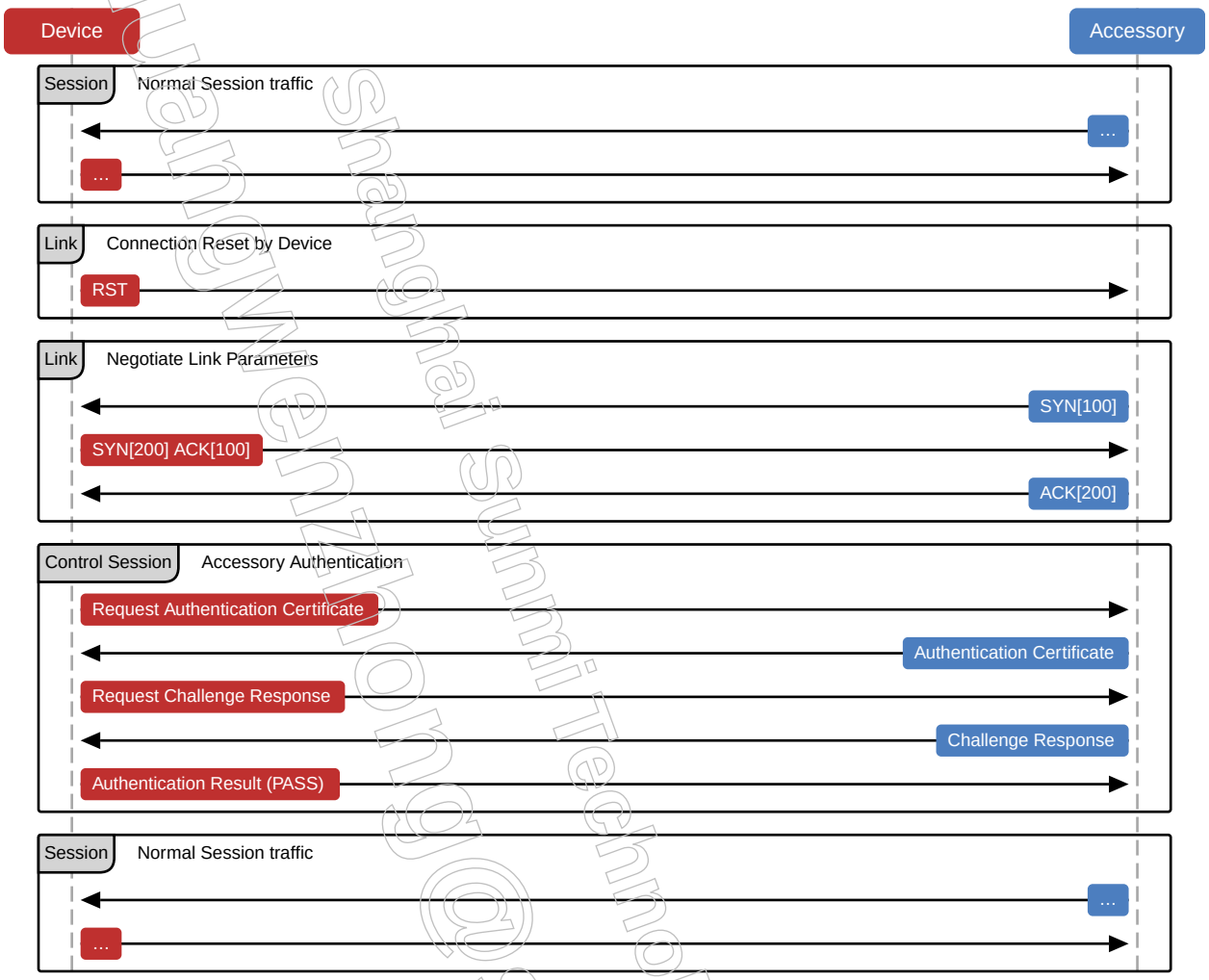
64.2.8.5 Normal Connection Traffic

After initialization/setup of the connection with an accessory, normal runtime traffic consists of regular ACK packets with data as needed.

At any time, a RST packet may be sent by the device to an accessory. The accessory shall reset the connection and start the connection sequence again by sending the SYN packet.

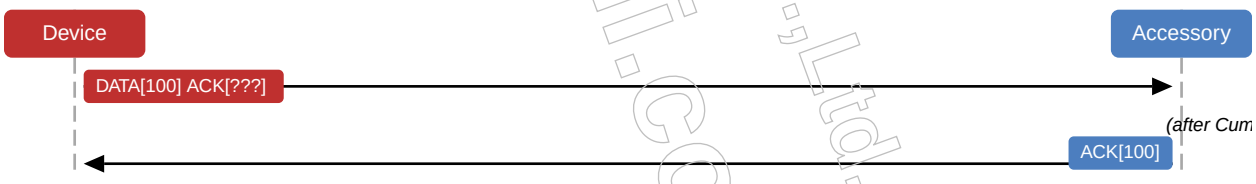


64.2.8.6 Device Reset of Transport Connection



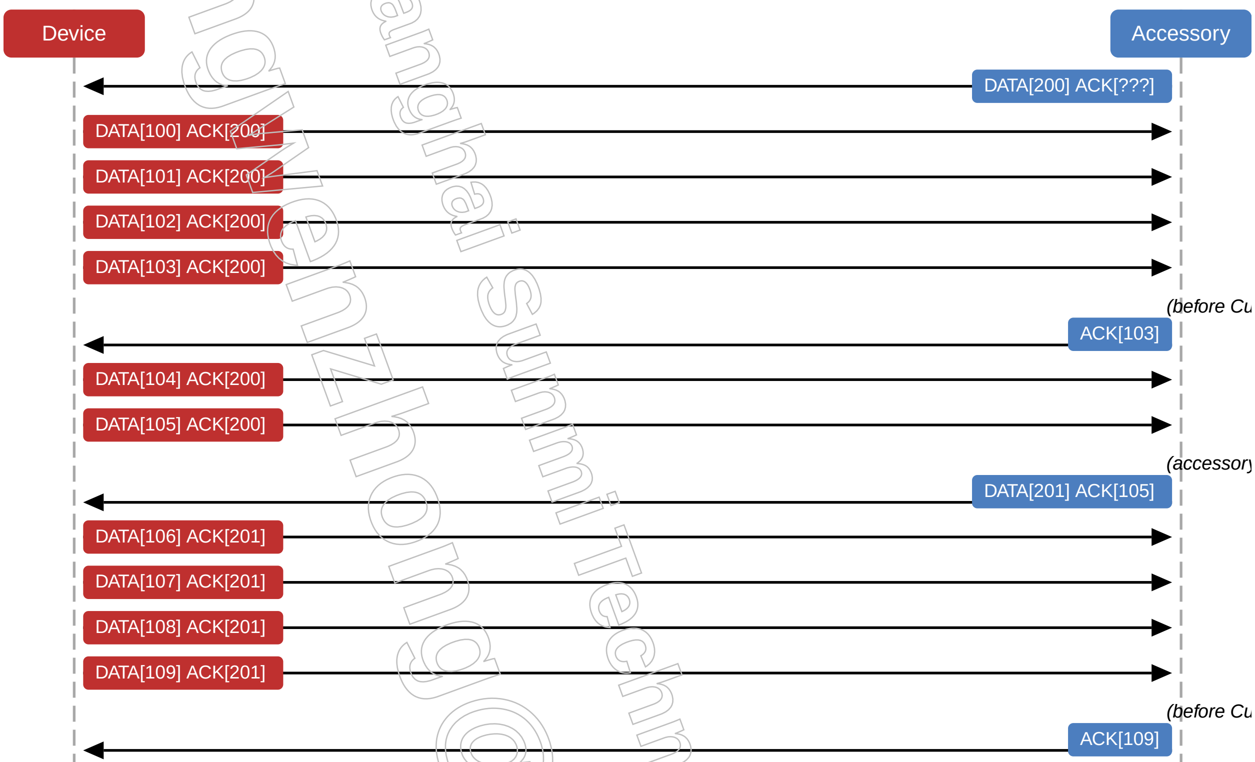
64.2.8.7 Cumulative Ack Timeout Expired

In this example, the Maximum Cumulative Acknowledgements link parameter is set to two. The device has sent one packet but has no need to send another one at this time. After the Cumulative Acknowledgement Timeout has expired the accessory assumes there is no need to wait for a second DATA packet and sends an ACK packet.



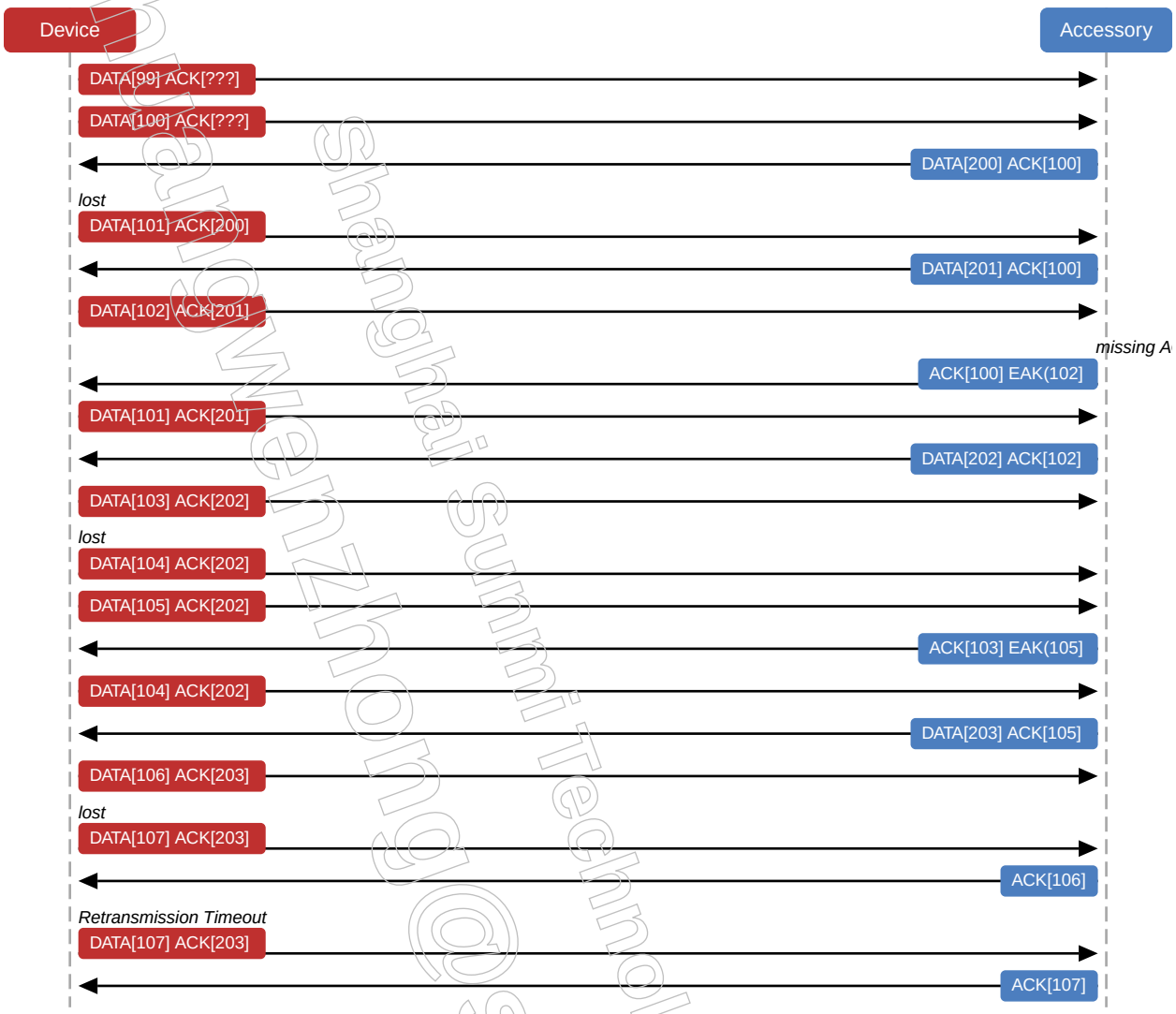
64.2.8.8 Continuous Data Transmission with ACKs

ACK packets (and Data packets with ACK) acknowledge receipt of a previously sent packet. In this example, the Maximum Cumulative Acknowledgement parameter is set to 4, and the device does not put more than 4 packets in flight at any given time. The accessory does not need to wait for Cumulative Acknowledgement Timeout to expire if it has additional packets to send.



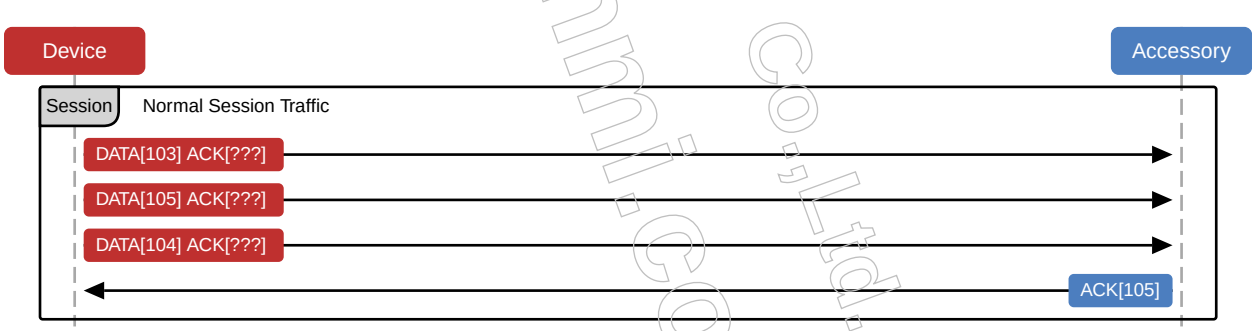
64.2.8.9 Resend of missing packets using EAK

An EAK packet is used to indicate missing packets. Upon receiving an EAK packet, the recipient shall resend the missing packets. The packet transfer recipient shall wait for acknowledgement of all in-flight packets up to the Maximum Cumulative Acknowledgement parameter, or wait the Cumulative Acknowledgement Timeout to expire before sending an EAK.



64.2.8.10 Receiving Packets Out Of Order

The accessory will receive packets out of order and ACK the last part of the data it receives.



64.2.8.11 iAP2 Link Packet Structure Example

Packet Length MSB and *Packet Length LSB* refer to the size of the packet (`0x001A`). *Control Byte* identifies the type of payload being sent (`0x80`), SYN packet. *Packet Sequence Number* in this case in randomly picked (`0x2B`). *Packet Acknowledgement Number* and *Session Identifier* have no meaning in this context. *Header Checksum* is calculated to be `0xE2`. *Payload Data* and *Payload Checksum* are described in detail in iAP2 Link Synchronization Payload Example (page 474).

See Packet Structure (page 454) for more details.

Table 64-11 iAP2 Link Packet Structure Example - Accessory SYN Packet

Start of Packet MSB (`FF`**)**
Start of Packet LSB (`5A`**)**
Packet Length MSB (`00`**)**
Packet Length LSB (`1A`**)**
Control Byte (`80` **SYN Packet)**
Packet Sequence Number (`2B`**)**
Packet Acknowledgement Number (`00`**)**
Session Identifier (`00`**)**
Header Checksum (`E2`**)**
Payload Data
(`01 05 10 00 04 0B 00 17 03 03 0A 00 01 0B 02 01`**)**
Payload Checksum (`A5`**)**

64.2.8.12 iAP2 Link Synchronization Payload Example

See Link Synchronization Payload (page 457) for more details.

Table 64-12 Link Synchronization Payload Example

Link Version (01)
Maximum Number of Outstanding Packets (05)
Maximum Received Packet Length MSB (10)
Maximum Received Packet Length LSB (00)
Retransmission Timeout MSB (04)
Retransmission Timeout LSB (0B)
Cumulative Acknowledgement Timeout MSB (00)
Cumulative Acknowledgement Timeout LSB (17)
Maximum Number of Retransmissions (03)
Maximum Cumulative Acknowledgements (03)
iAP2 Session 1: Session Identifier (0A)
iAP2 Session 1: Session Type (00)
iAP2 Session 1: Session Version (01)
iAP2 Session 2: Session Identifier (0B)
iAP2 Session 2: Session Type (02)
iAP2 Session 2: Session Version (01)

64.3 iAP2 Sessions

Once an iAP2 link has been established, higher level communication between an accessory and a device is structured into one or more sessions. At the very minimum, all accessories shall establish a control session for the purposes of authentication and identification. Once initial authentication and identification are complete, the accessory may use additional sessions.

Every session has a session identifier, session type, and session version. The session identifier is used to uniquely identify a session within an iAP2 connection. The session identifier of 0 is reserved for link use, and all sessions used by an accessory shall have unique nonzero identifiers. The combination of both session type and session version dictates which session protocol variant is used for all data traffic in the session. The session identifier is part of every iAP2 link packet header.

64.3.1 Attributes

64.3.1.1 Type

Table 64-13 iAP2 Session Types

Session Type	Name
0	Control session
1	File Transfer session
2	External Accessory session

All link implementations shall have one control session. All other sessions are optional depending on the features implemented. Multiple sessions of the same type are not permitted.

64.3.1.2 Version

The session version is used to distinguish between different variants of a particular session protocol. The exact nature of these differences depends on the session type. See the sections in the specification on each session type for more details.

64.3.2 Control Session

The control session has 2 primary goals:

- Notify both the device and accessory of changes in the other's state and/or configuration.
- Provide the accessory with means to request changes in device state and/or configuration.

The control session protocol is a stateless protocol. Each message is an independent entity with no relationship to any prior message. The device does not retain any information or status about each attached accessory from message to message. An accessory shall behave the same way. Specifically, an accessory request for a change in device state and/or configuration shall be treated as a request and not a command. The device may or may not choose to honor the request, and will either send an update message to confirm a change in state/configuration or not send any message at all.

Accessories supporting control session version 1 shall complete Accessory Authentication (page 276) before sending any other messages.

Accessories supporting control session version 2 may send Accessory Identification (page 281) messages before authentication completes and shall complete authentication before sending any other messages. The accessories shall fall back to control session 1 if the device does not support control session 2 or does not support those devices.

USB Host Mode (page 510) accessories shall not use control session version 2.

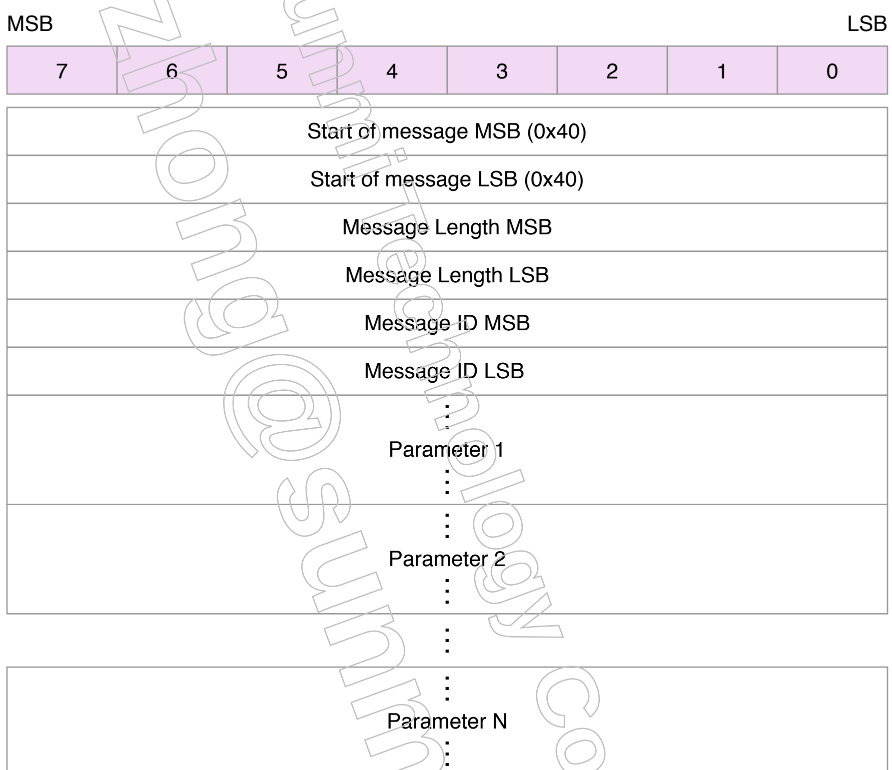
Accessories shall use control session messages for their specified purpose. Relying on unspecified behavior is likely to cause incompatibilities with future devices and/or iOS releases.

Accessories shall rely on the iAP2 link layer to guarantee message delivery. Accessories should never resend messages at the control session layer.

64.3.2.1 Message Structure

Each message has a header followed by 0 or more parameters or parameter groups:

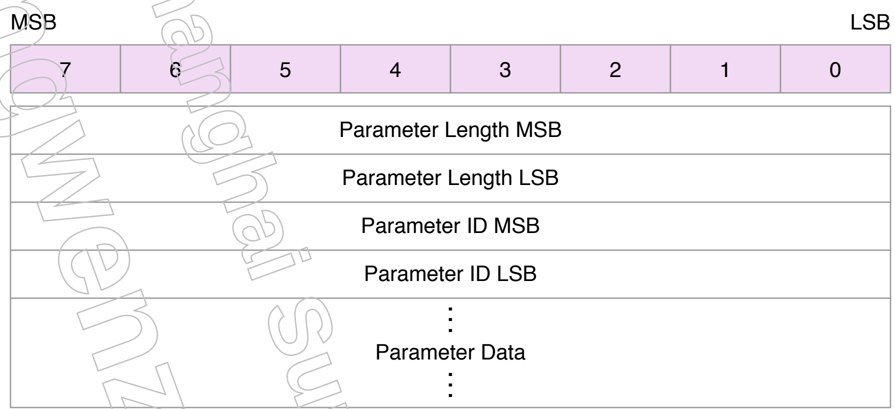
Figure 64-1 Control Session Message Structure



Message Length:
Size of the whole message, in bytes, including the Start of Message field, Message Length field, Message ID field and all Parameters.
Message ID:
Message Identifier
Parameter:
Parameters for this message (see below)

Parameters are structured as follows:

Figure 64-2 Control Session Message Parameter Structure



Parameter Length:
Length of parameter, in bytes, including the parameter length field, parameter ID and parameter data.
Parameter ID:
Interpretation is message specific.
Parameter Data:
Type of the data is determined by the parameter ID for this specific message ID.

64.3.2.2 Message Parsing

All iAP2 control session message parsers shall comply with the following requirements:

- Parameters can be specified in any order.
- Received messages shall be ignored if the message identifier is unrecognized.
- Received messages shall be ignored if any required parameters are missing.
- Received messages shall be ignored if any parameters have invalid values.
- Extra unknown parameters in a received message shall be ignored and message parsing shall proceed with the remaining parameters.

64.3.2.3 Parameter Types

All parameter bytes are transmitted and received in big-endian order.

64.3.2.3.1 Number

Only certain types of numbers are permitted in messages:

- **int8** - signed 8-bit byte
- **int16** - signed 16-bit word
- **int32** - signed 32-bit long
- **int64** - signed 64-bit quadword
- **uint8** - unsigned 8-bit byte
- **uint16** - unsigned 16-bit word
- **uint32** - unsigned 32-bit long
- **uint64** - unsigned 64-bit quadword

If a number parameter has a limited range of permitted values, the range will be specified in the parameter notes. Otherwise, the full range of possible values is assumed to be valid.

64.3.2.3.2 Duration

Duration parameters represent a period of time encoded as an unsigned integer. The unit of time and integer size is encoded in the type name.

The following duration parameter types are defined:

- **secs16** - seconds, unsigned 16-bit
- **secs32** - seconds, unsigned 32-bit
- **secs64** - seconds, unsigned 64-bit
- **msecs16** - milliseconds, unsigned 16-bit
- **msecs32** - milliseconds, unsigned 32-bit
- **msecs64** - milliseconds, unsigned 64-bit
- **usecs16** - microseconds, unsigned 16-bit
- **usecs32** - microseconds, unsigned 32-bit
- **usecs64** - microseconds, unsigned 64-bit

64.3.2.3.3 Enumeration

Written in parameter lists as 'enum'. Unsigned 8-bit byte which corresponds to an exact listing of all of its possible values.

64.3.2.3.4 Boolean

Written in parameter lists as 'bool'. Special type of Enumeration encoded as a one byte unsigned integer where 0 indicates NO and 1 means YES. All other enumeration values are invalid.

64.3.2.3.5 Array

Written in parameter lists as a number type followed by the number of expected elements in brackets. For example 'uint64[4]' indicates 4 'uint64' types arranged sequentially. Arrays can also be defined as having at least a certain number of elements, such as 'int8[1+]' to indicate the array shall have at least 1 element, but may have more.

Arrays assume an explicit meaning to the order of the elements and can be referenced by their 0-based index. So the first element is at index 0, the second is at index 1, and so on. This enables other parameters to be defined referring explicitly to an element in an array of a different parameter. For example, if a parameter 'ExampleArray' were defined of type 'uint64[4+]', a parameter 'DefaultValueIndex' could be defined of type 'uint8'. If the value of 'DefaultValueIndex' was set to '2' this would mean the parameter is referring to the third element of 'ExampleArray'.

64.3.2.3.6 Rational

Rational numbers are expressed as an array with two elements. The array element at index 0 is the numerator. The array element at index 1 is the denominator. The underlying number type used for the array is part of the type name.

The following rational number types are defined for use in messages:
- **rat16** - int16[2]
- **rat32** - int32[2]
- **urat16** - uint16[2]
- **urat32** - uint32[2]

64.3.2.3.7 String

Written in parameter lists as 'utf8'. Null terminated UTF-8 string. Accessories receiving string parameters shall be prepared to handle any valid UTF-8 string, as devices allow users to input strings in all supported languages and character sets, such as Emoji.

Decoding errors shall be indicated by the Unicode replacement character (U+FFFD, a question mark inside a diamond).

Unsupported characters shall be indicated by one of the following:
- White square (U+25A1).
- White vertical rectangle (U+25AF).
- Geta mark (U+3013).

64.3.2.3.8 Blob

Written in parameter lists as 'blob'. The contents of the binary blob will be defined in the parameter notes. Typically, most blobs are arrays of custom data structures. Accessories shall be prepared to handle zero-length blobs.

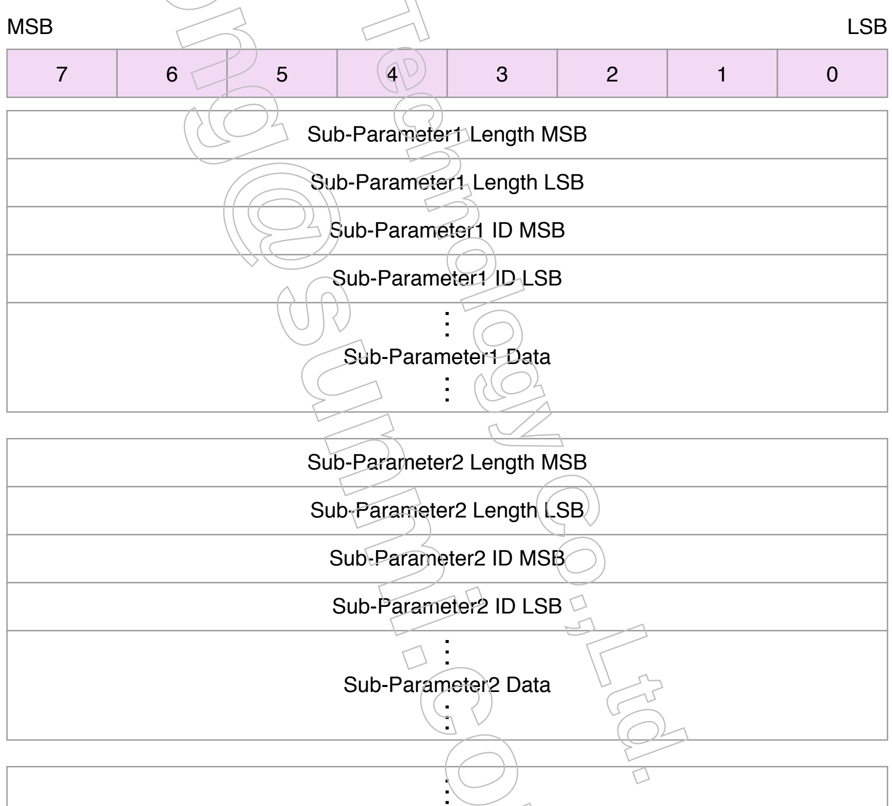
64.3.2.3.9 None

Written in parameter lists as 'none'. No parameter data. The mere existence or absence of the parameter in the message has meaning. Void parameters shall have a length of 4 and no payload data.

64.3.2.3.10 Group

Written in parameter lists as 'group'. Parameters may be grouped together as a group. Group parameter data consists of a number of sub-parameters each having a Format consisting of Sub-parameter Length, ID, and Data. Sub-parameters may be present in any order within a group.

Figure 64-3 Group Parameter Structure



64.3.2.4 Message Example

Example StartExternalAccessoryProtocolSession Control Session message

Figure 64-4 StartExternalAccessoryProtocolSession Control Session Message Example

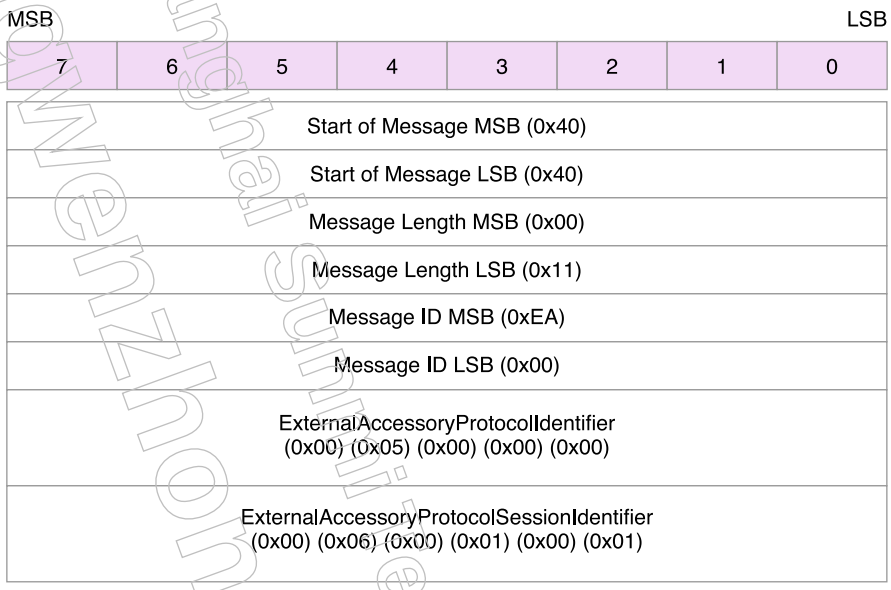


Figure 64-5 ExternalAccessoryProtocolIdentifier Parameter

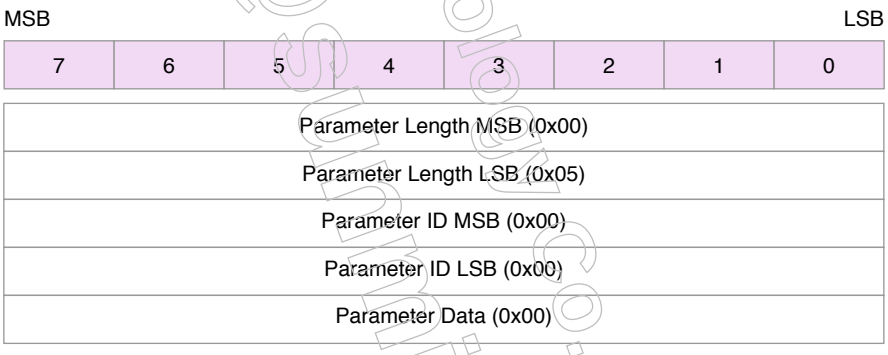


Figure 64-6 ExternalAccessoryProtocolSessionIdentifier Parameter

MSB LSB

7	6	5	4	3	2	1	0

Parameter Length MSB (0x00)
Parameter Length LSB (0x06)
Parameter ID MSB (0x00)
Parameter ID LSB (0x01)
Parameter Data (0x00) (0x01)

Example StartNowPlayingUpdates Control Session message with parameter group

Figure 64-7 StartNowPlayingUpdates Control Session Message Example

MSB LSB

7	6	5	4	3	2	1	0

Start of Message MSB (0x40)
Start of Message LSB (0x40)
Message Length MSB (0x00)
Message Length LSB (0x0E)
Message ID MSB (0x50)
Message ID LSB (0x00)
PlaybackAttributes (0x00) (0x08) (0x00) (0x01) (0x00) (0x04) (0x00) (0x0C)

Figure 64-8 PlaybackAttributes Parameter Group

MSB LSB

7	6	5	4	3	2	1	0

Sub-Parameter1 Length MSB (0x00)
Sub-Parameter1 Length LSB (0x08)
Sub-Parameter1 ID MSB (0x00)
Sub-Parameter1 ID LSB (0x01)
Sub-Parameter1 Data: PlaybackAttributes (0x00) (0x04) (0x00) (0x0C)

64.3.3 File Transfer Session

Any accessory identifying itself as capable of sending or receiving any messages associated with file transfers shall set up a file transfer session during link negotiation.

All file transfers are initiated by the sender and have a unique identifier also assigned by the sender. Up to 128 simultaneous transfers in each direction are possible, and transfers may be paused and/or canceled before they are complete by either the sender or the receiver.

All file transfer datagrams shall fit within a single iAP2 link packet payload.

At this time, the only valid file transfer session versions are 1 and 2.

Accessories should implement file transfer session version 2. Accessories supporting Media Library Playlist Transfers (page 398) of the Media Library Access (page 397) feature and/or Playback Queue List Transfers (page 413) of the Now Playing Updates (page 412) feature shall advertise support for file transfer session version 2.

File transfer session version 2 does not require a file transfer being set up in the control session. Instead, a Setup datagram contains information necessary to identify the file being transferred as described in Setup Datagram (Version 2) (page 485).

64.3.3.1 TransferIdentifier

Setup of a file transfer takes place in the control session. The following iAP2 messages can generate a file transfer identifier parameter:

- MediaLibraryUpdate (page 904)
- NowPlayingUpdate (page 911)

To support file transfers from the accessory to the device, every iAP2 connection shall maintain a global unsigned 8-bit FileTransferIdentifier starts at 0 when the connection is established. The FileTransferIdentifier shall be incremented to the next inactive FileTransferIdentifier value once it has been used. Upon the global FileTransferIdentifier reaching 127 the accessory shall immediately reset it to 0 or the lowest inactive FileTransferIdentifier value, whichever is lower, before it initiates the next transfer.

MediaLibraryUpdate and NowPlayingUpdate messages can generate a FileTransferIdentifier parameter for transfers from the device to the accessory, which will have parameters ranging from 128 to 255.

64.3.3.2 Setup Datagram (Version 1)

In file transfer session version 1, once a message with a FileTransferIdentifier parameter has been sent, the sender shall issue a Setup datagram containing the file size in bytes over the file transfer session using the same FileTransferIdentifier.

Table 64-14 File Transfer Session Setup Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x04`
2	File Size Byte 0
3	File Size Byte 1
4	File Size Byte 2
5	File Size Byte 3
6	File Size Byte 4
7	File Size Byte 5
8	File Size Byte 6
9	File Size Byte 7

64.3.3.3 Setup Datagram (Version 2)

In file transfer session version 2, once a message with a FileTransferIdentifier parameter has been sent, the sender shall issue a Setup datagram containing the file size in bytes followed by the file type and the file type setup data in bytes over the file transfer session using the same FileTransferIdentifier.

Table 64-15 File Transfer Session Setup Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x04`
2	File Size Byte 0
3	File Size Byte 1
4	File Size Byte 2
5	File Size Byte 3
6	File Size Byte 4
7	File Size Byte 5
8	File Size Byte 6
9	File Size Byte 7
10	File Type Byte 0
11	File Type Byte 1
12	File Type Setup Data Byte 0 (Depends on Type)
13	File Type Setup Data Byte 1 (Depends on Type)
…	…

Byte	Value
n+12	File Type Setup Data Byte n (Depends on Type)

File Types and related File Type Setup Data take the following values for transfer data.

Table 64-16 Setup Datagram File Types and File Type Setup Data

File Type	File Type Name	File Type Setup Data	Transfer Data
`0x0000`	Reserved	Invalid	
`0x0002`	NowPlayingArtworkData	None	JPEG data
`0x0003`	NowPlayingPlaybackQueueContents	None	Item PIDs in uint64[]
`0x0004`	MediaLibraryUpdatePlaylistContents	PlaylistPID (uint64) + LibraryUID	Item PIDs in uint64[]
`0x0006`	MediaItemListNowPlaying-PlaybackQueueContents	None	See MediaItemList (page 486).
`0x0007`	MediaItemListMediaLibraryUpdate-PlaylistContents	PlaylistPID (uint64) + LibraryUID	See MediaItemList (page 486).
`0x0008`	AppDiscoveryIconData	AppBundleID (utf8)	PNG data

Table 64-17 `MediaItemList` transfer data

Name	ID	Type	#	Notes
MediaItemCount	0	uint32	1	
MediaItemStartIndex	1	uint32	1	
MediaItem	2	group	0+ (ordered)	See below.

The MediaItem parameter of MediaItemList (page 486) is an ordered list of MediaItem (page 905) information formatted using the same message parameter format as for MediaLibraryUpdate (page 904) and NowPlayingUpdate (page 911). Unlike the iAP2 control session message format, the items shall be in order. The valid parameters of each MediaItem are specified by MediaPlaylistContentTransferInfoRequest (page 903) for MediaLibraryUpdate (page 904), or PlaybackQueueListContentTransferInfoRequest (page 911) for NowPlayingUpdate (page 911).

64.3.3.4 Start

Upon receiving the Setup datagram from the sender, the receiver may send a Start datagram back to the sender indicating the receiver is ready to receive file data.

Table 64-18 File Transfer Session Start Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x01`

64.3.3.5 FirstData

The sender may start sending file data once it receives a Start datagram. The first datagram is unique and shall take the following format:

Table 64-19 File Transfer Session FirstData Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x80`
2	Data Byte 0
...	...
n+2	Data Byte n

Only one FirstData datagram may be sent for any given assigned FileTransferIdentifier. The sender shall switch to Data datagrams for all subsequent file data until the end of the file.

64.3.3.6 FirstAndOnlyData

If the file is so short as to fit in one session datagram, the sender may send a FirstAndOnlyData datagram as follows:

Table 64-20 File Transfer Session FirstAndOnlyData Datagram

Byte	Value
0	FileTransferIdentifier
1	`0xC0`
2	Data Byte 0
...	...
n+2	Data Byte n

Only one FirstAndOnlyData datagram may be sent for any given assigned FileTransferIdentifier. The file transfer is considered complete upon receipt of the FirstAndOnlyData datagram.

64.3.3.7 Data

Regular file data datagrams shall take the following format:

Table 64-21 File Transfer Session Data Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x00`
2	Data Byte 0
…	…
n+2	Data Byte n

64.3.3.8 LastData

The last object data datagram is unique and shall take the following format:

Table 64-22 File Transfer Session LastData Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x40`
2	Data Byte 0
…	…
n+2	Data Byte n

64.3.3.9 Cancel

The Cancel datagram may be sent at any time by either the sender or receiver after the Setup datagram is sent or received. The FileTransferIdentifier shall be marked as inactive by both the sender and receiver after it is transmitted.

The Cancel datagram shall be sent by any accessory or device if any other File Transfer Session datagram is received with an inactive FileTransferIdentifier.

Table 64-23 File Transfer Session Cancel Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x02`

64.3.3.10 Pause

The Pause datagram may be sent at any time by the sender or receiver after the first Start datagram has been sent by the receiver. Unlike the Cancel datagram, the FileTransferIdentifier remains active, but no FirstData, Data, or LastData datagrams may be transmitted until the end sending the Pause datagram sends a Start datagram with the same FileTransferIdentifier. Data datagrams will resume from the point where the transfer left off.

Table 64-24 File Transfer Session Pause Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x03`

64.3.3.11 Success

The Success datagram shall be sent by the receiver after it has received and processed all file data. The sender may choose to delete the file or otherwise dispose of it after receiving this datagram from the receiver.

The FileTransferIdentifier shall be marked as inactive by both the sender and receiver after this datagram is transmitted.

Table 64-25 File Transfer Session Success Datagram

Byte	Value
0	FileTransferIdentifier
1	`0x05`

64.3.3.12 Failure

The Failure datagram shall be sent by the receiver if it has received all file data but is unable to store it. This sender may choose to resend the file or not, depending on the circumstances.

The FileTransferIdentifier shall be marked as inactive by both the sender and receiver after this datagram is transmitted.

Table 64-26 File Transfer Session Failure Datagram

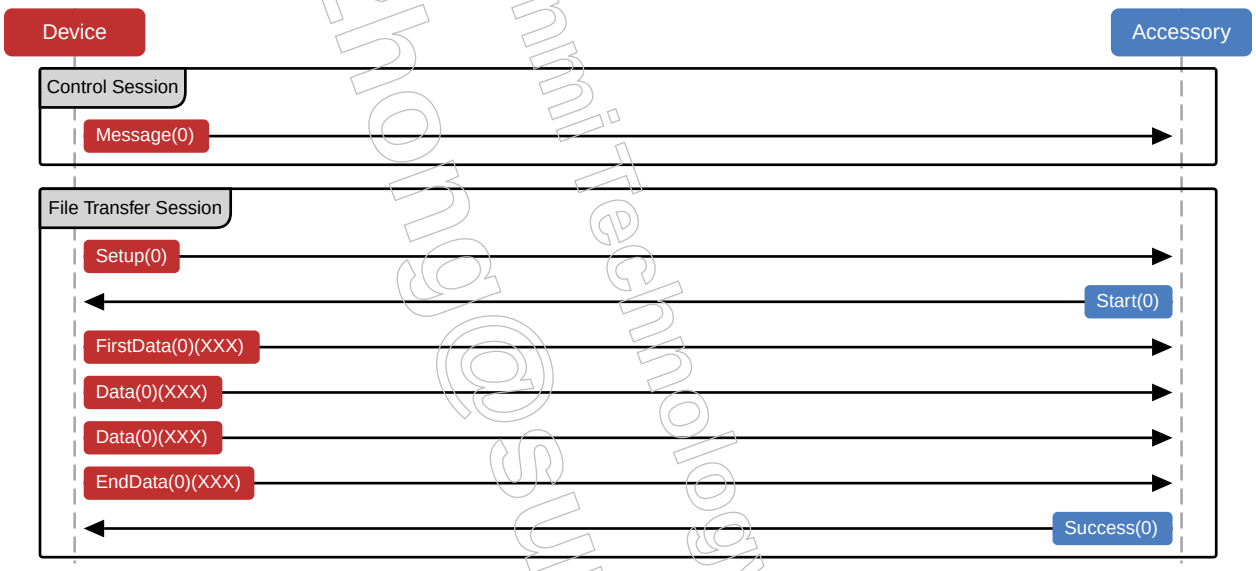
Byte	Value
0	FileTransferIdentifier
1	0x06

64.3.3.13 Examples

In all examples, the device is playing the role of the file sender, and the accessory is the file receiver. These roles can be reversed depending on the needs of a particular accessory interface feature

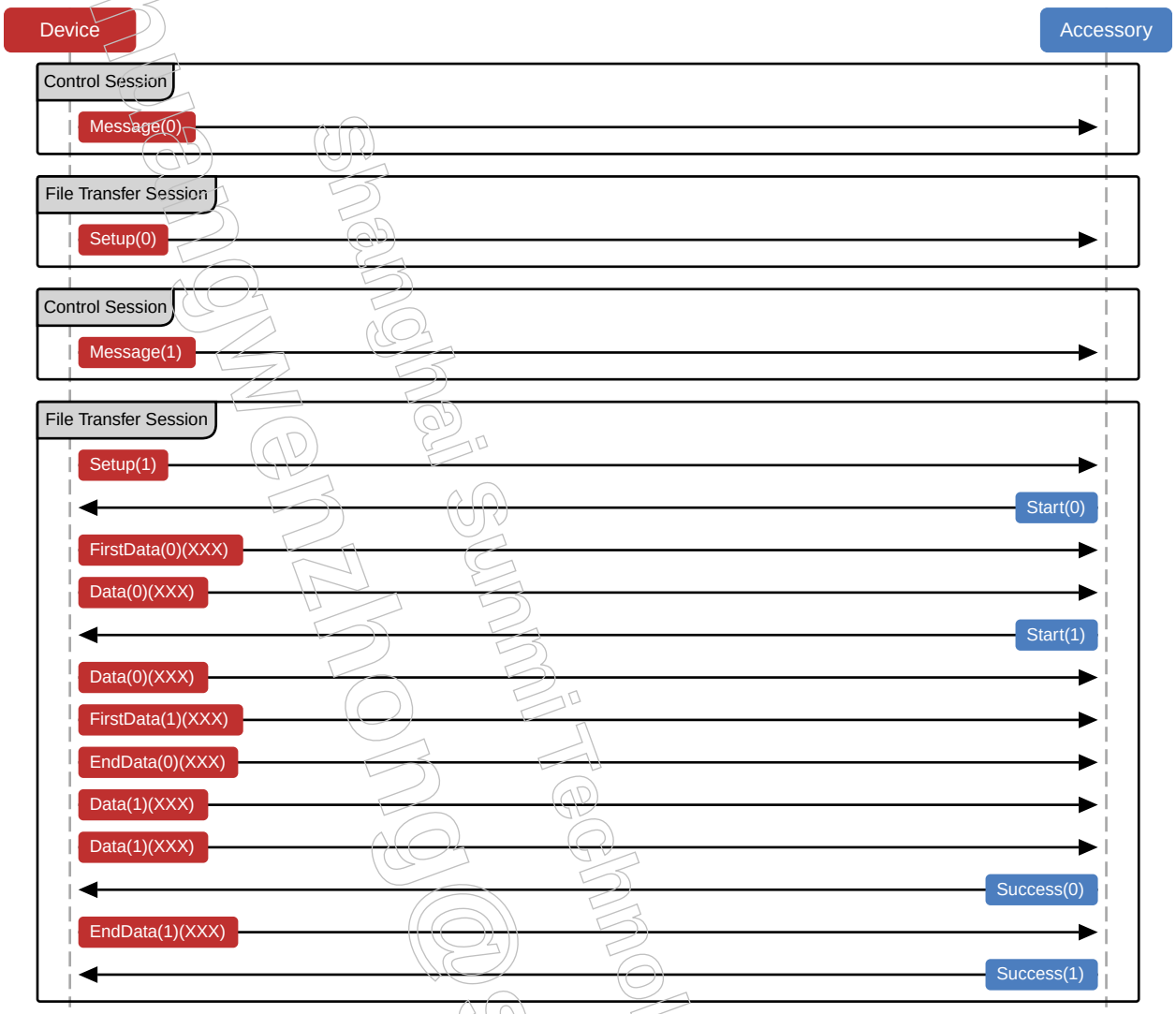
64.3.3.13.1 Typical Transfer

In this example, the device is sending one file to the accessory, and no other transfers are taking place.



64.3.3.13.2 Two Simultaneous Transfers

In this example, the device is sending two files to the accessory simultaneously. The traffic can be interleaved arbitrarily, so long as the message containing the FileTransferIdentifier is sent before any file transfer session traffic starts.



64.3.4 External Accessory Session

The external accessory (EA) session is used by the External Accessory Protocol (page 352) to create one or more bidirectional data streams, or transfers, between an accessory and iOS apps.

At this time, the only valid external accessory session version is 1.

64.3.4.1 Setup

Setup of an EA session takes place in the control session. The accessory will receive a StartExternalAccessoryProtocolSession (page 896) message with assigned `ExternalAccessoryProtocolIdentifier` and `ExternalAccessoryProtocolSessionIdentifier` parameters.

The `ExternalAccessoryProtocolIdentifier` is defined by the accessory during identification; an accessory may define and support multiple supported protocols.

The `ExternalAccessorySessionIdentifier` is unique to each EA connection between an app and the accessory. Accessories shall use this identifier to distinguish between datagrams sent over the EA session.

Accessories declaring an EA session during link synchronization shall declare support for one or more `ExternalAccessoryProtocol` parameters during accessory identification, see External Accessory Protocol (page 352).

Accessories shall not send any EA session datagrams with an unassigned transfer identifier. Conversely, accessories shall ignore any EA session datagrams with unassigned session identifiers.

64.3.4.2 ExternalAccessorySession Datagram

All datagrams for an EA session shall follow this format:

Table 64-27 ExternalAccessorySession Datagram

Byte	Value
0	ExternalAccessorySessionIdentifier Byte 0
1	ExternalAccessorySessionIdentifier Byte 1
2+	External Accessory Session Data

All EA session datagrams shall fit within a single iAP2 link packet payload.

64.4 Test Procedures

64.4.1 Link Layer (v1)

1. Verify accessories (except A2M accessories) use only Link Version number 1.
2. With devices running iOS 7.0 or later, verify the Link Layer session starts with the following sequence:
   - Accessory DETECT
   - Device DETECT
   - Accessory SYN
   - Device SYN+ACK
   - Accessory ACK
   - Device ACK RequestAuthenticationCertificate
   - Accessory ACK AuthenticationCertificate

- Device ACK RequestAuthenticationChallengeResponse
- Accessory ACK AuthenticationResponse
- Device ACK AuthenticationSucceeded
- Device ACK StartIdentification
- Accessory ACK IdentificationInformation
- Device ACK IdentificationAccepted
- Accessory ACK
- Additional ACKs packets with no payload (length of 9) are acceptable.

3. Verify the Packet Sequence Number increments by 1.
4. Verify all ACK packets without payloads (length of 9) use Session Identifier 0.
5. Verify all ACK packets with payloads use a non-zero Session Identifier.

64.4.2 Control Session

1. Verify accessories exhaustively enumerate all iAP2 control session messages they can send or receive.
2. Authentication and Identification messages may not be listed as supported features in IdentificationInformation; all accessories use them. Some claimed messages may appear; exceptions include DeviceHIDReport, StopHID, StopPowerUpdates, StopNowPlayingUpdates, StopUSBDeviceModeAudio and StopAssistiveTouch.
3. Verify fields (for example, product name, brand name) are correctly populated without "filler" entries (for example, "ACME", "1234").